



Technical Document

Manage Workflow — SDD

SOFTWARE DESIGN DOCUMENT
Manage Workflow · Workflow Designer

Software Design Document (SDD) — Manage Workflow

Feature: Manage Workflow (Workflow designer) **Apex controller:** force-app/main/default/classes/controller/manageWorkflow/ManageWorkflowPageController.c
LWC: force-app/main/default/lwc/manageWorkflow/ **Date:** 2026-06-22

Image references below follow the marker convention used across these design documents: the image file name is wrapped between ##### lines, immediately before the screenshot belongs. This document ships without the screenshots — so each marker pair encloses a one-line comment describing the image that should be placed there instead of an embedded image.

1. Architecture Overview

```
manageWorkflow (parent LWC) —
Principal state: workflowData = {
  id, projectStatus:[ { id, name } ],
  workflow:{ id, name, transitions:[
    { id, name, fromStatus, toStatus, recordStatus, validateFields[] } ] } }
UI-only state (outside principal): _projectId · _workflowId ·
  clickedStatusIds · selectedFromStatus/selectedToStatus ·
  selectedTransitionId · _isValidationDetailExpanded · _wiredTransitionId ·
  config (responsive) · modal flags (status/transition/workflow/validation)
Derived getters: sortedStatuses · statusPositions · statusesWithSVGData ·
  activeTransitionLines · pendingTransitionLines · svgViewBox ·
  activeTransition · validationFields · showTransitionDetail

workflowUtils.js (pure): getResponsiveConfig · gatherSortedStatuses ·
  calculatePositions · calculateTransitionLines · createArrowPath ·
  getRectIntersection · getSvgViewBox · getStatusesWithSVGData · toggleClick
workflowValidator.js (pure): validateStatusName · validateTransition ·
  validateTransitionName · validateValidationType · validateTicketField ·
  validateTransitionId

Children
<c-choose-project> (step 1, project picker)
workflow list (step 2): cards + Create-Workflow modal
SVG canvas (step 3): status rects · transition arrows (active/pending)
modals: create status · create transition · transition-detail panel ·
  add-validation-rule section
```

| imperative writes/reads + @wire(loadValidateFields, cacheable=true)

▼
ManageWorkflowPageController
→ WorkflowService · StatusService · ValidateFieldService · DomainCorrectness
→ Workflow__c · Status__c · WorkflowTransition__c · ValidateField__c · Project__c

Response envelope. Every controller method returns `APIResponse { success, message, data }`. The LWC treats `success === false` (or a thrown Apex error) as failure and toasts it via `_toast(...,"error")`; successful writes patch the principal state from the returned data (the persisted record), never from an optimistic draft.

Data shape used by the editor (from `getWorkflow` → `WorkflowConfigDTO`): `{ id, projectStatus:[{id,name}], workflow:{ id, name, transitions:[{id,name,fromStatus,toStatus,recordStatus}] } }`. A transition's `validateFields[]` are not part of this payload — they are loaded lazily per transition by `loadValidateFields`

and folded onto the owning transition inside the same principal state, so the canvas needs no further round-trip to draw the graph.

2. API Calls (Apex @AuraEnabled methods)

Legend: **C** = cacheable=true (wire-friendly), **I** = imperative (cacheable=false or DML). All methods return `APIResponse{success,message,data}`.

2.1 Workflow list & editor bootstrap

Method	Kind	Params	Returns (`data`)	LWC usage
<code>loadWorkflowsByProject</code>	C	projectId	Workflow__c[] (Id, Name, RecordStatus__c, dates)	<code>_loadWorkflowsForProject()</code> — called imperatively to build the list
<code>getWorkflow</code>	I	workflowId	WorkflowConfigDTO {projectStatus, workflow{transitions}}	<code>_loadWorkflow()</code> on entering the editor (cacheable=false)
<code>createWorkflow</code>	I	name, projectId	Workflow__c	Create-Workflow modal → then opens editor on the new id

2.2 Status (node) creation

Method	Kind	Params	Effect
<code>createStatus</code>	I	name, projectId	creates a project Status__c; returns the record → <code>_addStatus({id,name})</code>

2.3 Transition create / lifecycle

Method	Kind	Params	Effect
<code>addWorkflowTransition</code>	I	workflowId, name, fromStatusId, toStatusId	creates a WorkflowTransition__c in pending status; returns it → <code>_addPendingTransition(...)</code>
<code>activateWorkflowTransition</code>	I	workflowTransitionId	pending → active; LWC → <code>_setTransitionRecordStatus(id,"active")</code>
<code>deleteWorkflowTransition</code>	I	workflowTransitionId	soft delete (record status → deleted); LWC → <code>_removeTransition(id)</code>
<code>updateFullWorkflowTransition</code>	I	workflowId	activates all pending transitions of the workflow; returns the activated count

2.4 Validation rules

Method	Kind	Params	Effect
<code>loadValidateFields</code>	C	transitionId	ValidateField__c[] for the transition (wire, on expand)
<code>addValidateField</code>	I	workflowTransitionId, fieldName, type	creates a ValidateField__c; returns it → <code>_addTransitionValidateField(id,...)</code>

2.5 Wires actually bound in manageWorkflow.js

- @wire(loadValidateFields, { transitionId: '\$_wiredTransitionId' }) → maps each ValidateField__c and folds the list onto the open transition via _setTransitionValidateFields. The gate _wiredTransitionId stays undefined (wire dormant) until the user expands the detail, so selecting a transition does not by itself fetch its rules; collapsing clears the gate so the next expand re-fires.

All other interactions — the workflow list load, getWorkflow, and every write (createWorkflow, createStatus, addWorkflowTransition, activateWorkflowTransition, deleteWorkflowTransition, updateFullWorkflowTransition, addValidateField) — are imperative Apex calls inside handlers (then/catch/finally), toasting via _toast.

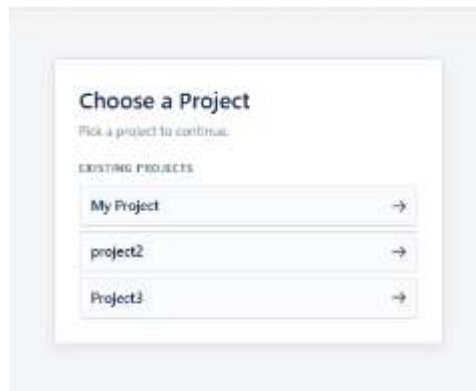
The controller also exposes endpoints this LWC does not bind (getProjectStatuses, loadWorkflows, loadStatuses, loadTransitions, getWorkflowTransitionById, getTicketTypeById, updateWorkflow). They are available for reuse but out of scope for the Manage Workflow page.

3. UI / UX Specification

3.1 Step 1 — project picker

If there is no projectId the page renders <c-choose-project>; on projectchosen the project is stored and the workflow list loads.

Manage-workflow-choose-project.png
#####



The Choose Project splash shown when no project is selected (the <c-choose-project> child).

3.2 Step 2 — workflow list

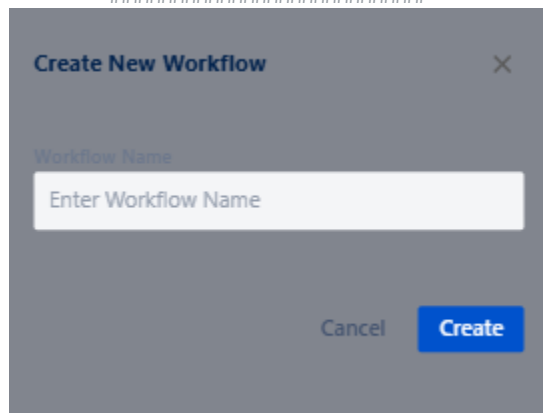
With a project but no workflow selected the page shows the “Workflows” heading, a Create New Workflow button, and one card per workflow (name + record status) each with an Edit button; an empty project shows “No workflows for this project yet — create one to get started.” The Create New Workflow modal takes a name and, on success, opens the editor on the new workflow.

Manage-workflow-list.png
#####



The workflow list: “Workflows” heading, “Create New Workflow” button, and workflow cards (name + record status + Edit).

Manage-workflow-create-modal.png
#####



The Create New Workflow modal: a Workflow Name input with Cancel / Create.

3.3 Step 3 — the visual editor (toolbar, legend, canvas)

The editor renders a toolbar (Back · Create Status · Update Workflow) and an info span that reads “Click two statuses to create a transition”, or “From: X → To: Y” once a source is selected. A legend distinguishes Active vs Pending transitions. The canvas is a single SVG whose viewBox is derived from the status count and the responsive config.

Manage-workflow-editor.png
#####

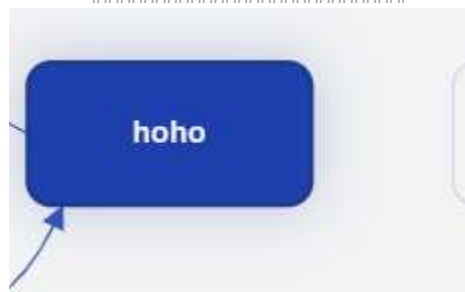


The editor: toolbar, Active / Pending legend, status nodes in a grid, and curved transition arrows (solid = active, dashed = pending) with name labels.

3.4 Status nodes & selection

Each status is a rounded rectangle labelled with the status name; clicking a node selects it (highlighted via `clickedStatusIds`). The first click sets the From status; clicking it again deselects; clicking a different node sets the To status and opens the Create Transition dialog. Create Status adds a new node.

Manage-workflow-status-selected.png
 #####

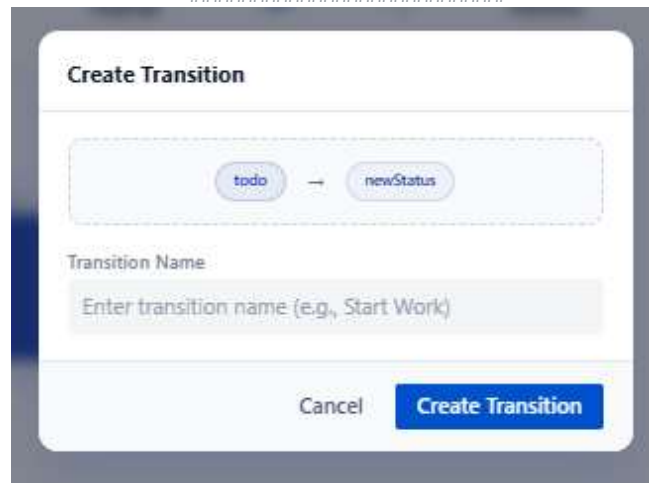


A status node selected as "From" (highlighted), with the toolbar info span showing "From: <status>".

4.5 Transition arrows (active vs pending)

Transitions are drawn as quadratic-Bézier curved arrows between node edges (`getRectIntersection` finds where the curve meets each rectangle; `createArrowPath` curves it to avoid overlapping nodes). Active transitions are solid; pending transitions are dashed. Each arrow carries the transition name at its mid-point and an explicit arrow-head polygon at the target end.

Manage-workflow-create-transition.png
#####

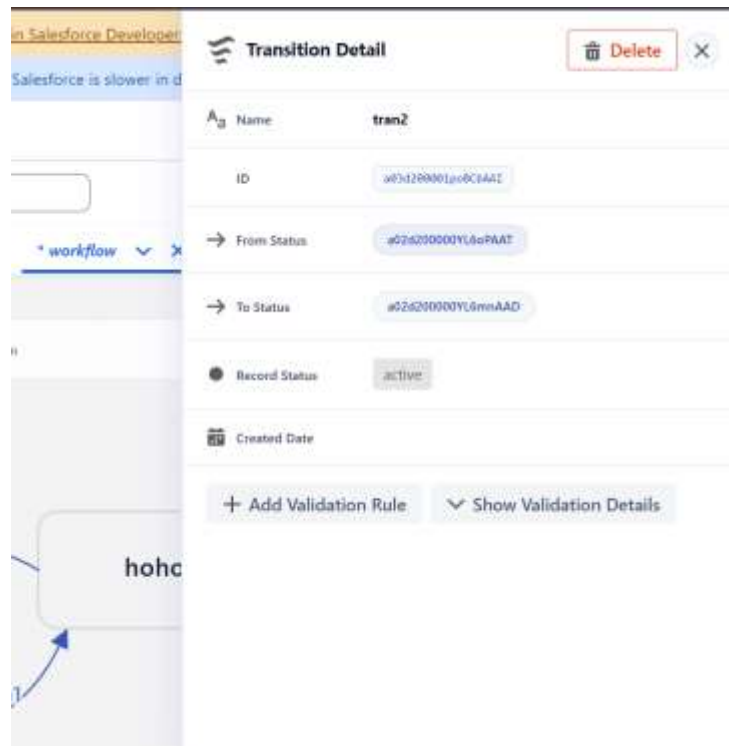


The Create Transition dialog after selecting two nodes: "From → To" badges and a Transition Name field with Cancel / Create Transition.

3.6 Transition detail panel

Clicking an arrow opens the detail panel: Name, Id, From / To status badges, Record Status badge, and Created Date. While the transition is pending an Activate action is shown; Delete (with a confirm prompt) soft-deletes it; Close dismisses the panel. The panel also hosts Add Validation Rule and Show / Hide Validation Details.

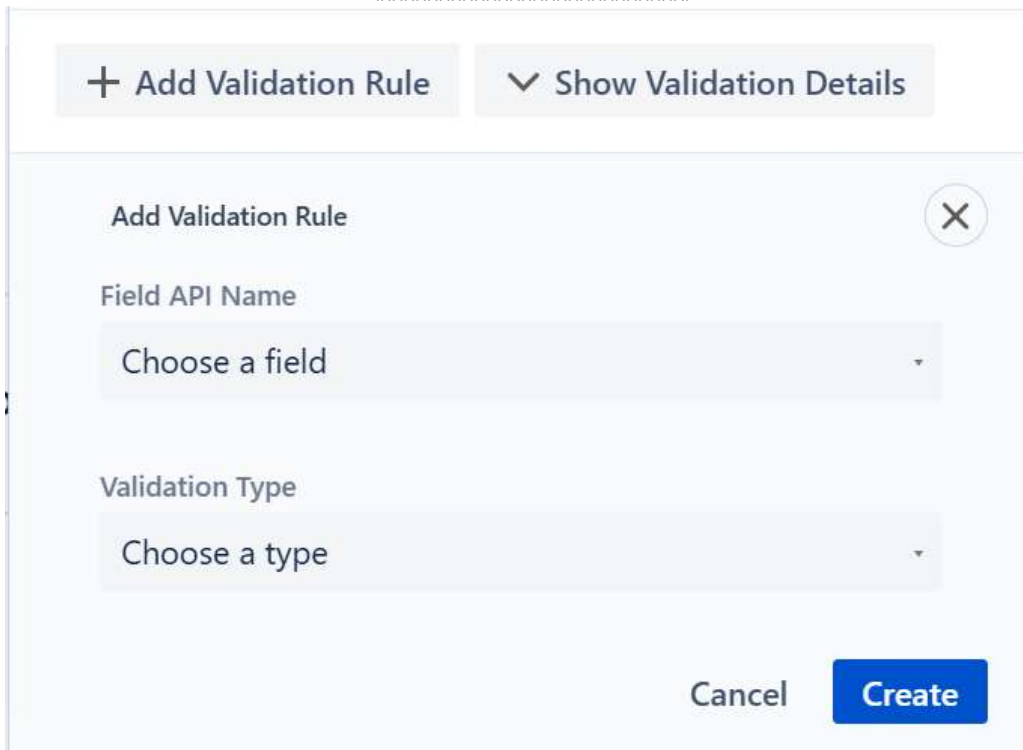
Manage-workflow-transition-detail.png
#####



3.7 Validation rules

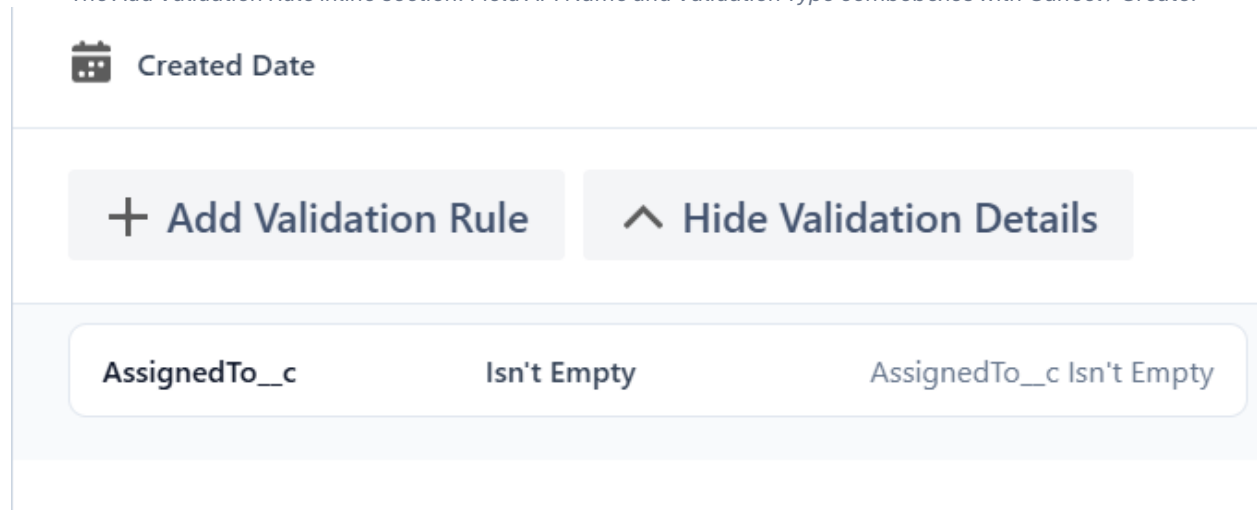
Add Validation Rule opens an inline section with a Field API Name combobox (mirroring the ValidateField__c field picklist) and a Validation Type combobox; Create persists the rule and folds it onto the transition. Show Validation Details expands the rule list (loaded on expand via loadValidateFields); an empty transition shows “No validation details for this transition.”

Manage-workflow-add-validation-rule.png
#####



The Add Validation Rule inline section: Field API Name and Validation Type comboboxes with Cancel / Create.

 Created Date



AssignedTo__c	Isn't Empty	AssignedTo__c Isn't Empty
---------------	-------------	---------------------------

A transition expanded to show its validation rules (field · type · error message rows).

4. Traceability (SRS → API)

SRS	API call(s) / mechanism
FR-1 select project	<c-choose-project> → localStorage projectId
FR-2/FR-3 workflow list & empty state	loadWorkflowsByProject → _workflows / empty-state getter
FR-4 create workflow	createWorkflow → _enterWorkflowEditor
FR-5/FR-6 open editor & load config	getWorkflow (WorkflowConfigDTO)
FR-7 back to list	handleBackToWorkflowList (+ ResizeObserver teardown)
FR-8 render status nodes	gatherSortedStatuses → statusesWithSVGData (derived)
FR-9 create status	createStatus → _addStatus
FR-10 responsive layout	getResponsiveConfig via ResizeObserver
FR-11/FR-12 select two statuses & dialog	handleStatusClick → openCreateTransitionModal
FR-13 pending transition	addWorkflowTransition → _addPendingTransition
FR-14/FR-16 arrows & labels	calculateTransitionLines (active vs pending sets)
FR-15 legend	static legend (active / pending)
FR-17 transition detail	handleTransitionClick → activeTransition (derived)
FR-18 activate	activateWorkflowTransition → _setTransitionRecordStatus
FR-19 delete	deleteWorkflowTransition (soft) → _removeTransition
FR-20 close panel	handleCloseTransitionDetail
FR-21 add validation rule	addValidateField → _addTransitionValidateField
FR-22 view validation rules	loadValidateFields (wire, on expand) → _setTransitionValidateFields
FR-23 update workflow	updateFullWorkflowTransition → _activatePendingTransitions